

1 . ТЕОРИЯ ФОРМАЛЬНЫХ ГРАММАТИК И ЯЗЫКОВ

План:

1. Грамматика.
2. Формальные определения грамматики и языка. 3. Классификация грамматик.
4. Синтаксические деревья.

Ключевые слова: синтаксис языка, семантика языка, грамматика, формальный язык, синтаксический язык.

1.1. ГРАММАТИКА

Грамматика языка является формальным описанием его синтаксиса или формы, в которой записаны отдельные предложения программы или вся программа. Грамматика не описывает семантику или значения различных предложений.

В качестве иллюстрации разницы между синтаксисом и семантикой рассмотрим два предложения:

$$i := j + k \text{ и } i := x + y ,$$

где x, y являются действительными переменными, а i, j, K целыми.

Эти два предложения имеют одинаковый синтаксис. Оба являются операторами присваивания. Но с точки зрения семантики, эти предложения разные, так как компилируются в совершенно различные последовательности машинных команд.

В первом случае складываются целые переменные j и K и результат присваивается переменной i .

Во втором случае необходимо сложить вещественные переменные x и y , а результат преобразовать к целому виду и полученное значение присвоить переменной i .

В качестве примера мы будем использовать программу на языке Паскаль, изображённую на рис. 1, однако обсуждаемые концепции и подходы приложимы и к другим языкам. Существует несколько различных форм записи грамматик, среди которых мы рассмотрим форму Бекуса—Наура (БНФ). БНФ не самое мощное из известных средств описания синтаксиса. Однако эта форма достаточно проста, широко используется и предоставляет достаточные для большинства приложений средства.

На рис. 2 изображена одна из возможных грамматик БНФ для очень узкого подмножества языка Паскаль.

```
program primer; var
sum,  a,  rez,  i:
integer;      begin
sum:=0; for i:=1 to
100  do  begin
read(a);
sum:=sum+a  end;
rez:=sum div100—
a*a; write(rez,sum)
end.
```

Рис. 1

Грамматика БНФ состоит из множества правил вывода, каждое из которых определяет синтаксис некоторой конструкции языка. Рассмотрим, например, правило 8 на рис. 2:

<присваивание> ид := <арифметическое
выражение>

1. <программа> <имя программы> var <раздел переменных> begin <раздел операторов> end.
2. <имя программы> ид
3. <раздел переменных> <список переменных>:<тип>
 4. <список переменных> ид /<список переменных>, ид
 5. <тип> integer
 6. <раздел операторов> —
><оператор>/<раздел операторов>;<оператор>
 7. <оператор>
<присваивание>/<ввод>/<вывод>/<цикл>
 8. <присваивание> ид :=
<арифметическое выражение>
 9. <арифметическое выражение>
<слагаемое>/<арифметическое выражение> +1-
<слагаемое>
 10. <слагаемое> <значение> / <слагаемое>
*/div <значение>
 11. <значение> ид/
конст/(<арифметическое выражение>)
 12. <ввод> read (<список переменных>)
 13. <вывод> write (<список переменных>)
 - 14.<щжл> for <выражение цикла> to <тело
цикла>
 15. <выражение цикла> ид :=<арифметическое
выражение> do <арифметическое выражение>

16. <тело цикла> <оператор>/ begin <раздел операторов> end

Рис. 2

Это определение оператора присваивания языка ПАСКАЛЬ, обозначенное как <присваивание>. Символ ”—>” можно читать как ”является по определению“. С левой стороны от этого символа находится определяемая конструкция языка (в нашем случае <присваивание>), а с правой — описание синтаксиса этой конструкции. Строки символов, заключенные в угловые скобки, называются нетерминальными символами. Они составляют синтаксические классы языка. Символы, не заключённые в угловые скобки, называются терминальными символами. Они составляют лексические классы или алфавит языка. В рассматриваемом примере нетерминальными символами являются <присваивание> и <арифметическое выражение>, а терминальными — ид и :=. Таким образом, это правило определяет, что конструкция <присваивание> состоит из идентификатора (терминал ид), символа алфавита :=, за которым следует конструкция <арифметическое выражение>. Пробелы для написания грамматики не существенны и вставляются только для наглядности.

Для распознавания нетерминального символа <присваивание> необходимо, чтобы существовало определение для нетерминального символа <арифметическое выражение>. Это определение даётся правилом 9 на рис. 2:

$$\langle \text{арифметическое выражение} \rangle \\ \langle \text{слагаемое} \rangle / \langle \text{арифметическое} \\ \text{выражение} \rangle + / - \langle \text{слагаемое} \rangle$$

Это правило предполагает две возможности, разделённые символом /. Первая состоит в том, что $\langle \text{арифметическое выражение} \rangle$ состоит из одной конструкции $\langle \text{слагаемое} \rangle$. Другой вариант заключается в том, что $\langle \text{арифметическое выражение} \rangle$ состоит из конструкции $\langle \text{арифметическое выражение} \rangle$, за которым следует знак + или —, за которым следует конструкция $\langle \text{слагаемое} \rangle$. Это правило является рекурсивным, т.е. конструкция $\langle \text{арифметическое выражение} \rangle$ определяется рекурсивно в терминах себя самой. В соответствии с этим правилом нетерминальным символом $\langle \text{арифметическое выражение} \rangle$ является любая последовательность из одного или более слагаемых, разделённых знаками + или —.

1.2. ФОРМАЛЬНЫЕ ОПРЕДЕЛЕНИЯ ГРАММАТИКИ ЯЗЫКА

Прежде, чем приступать к реализации транслирующих программ, необходимо определить некоторые понятия.

Алфавит языка — непустое, конечное множество символов. Например, пусть алфавит языка включает три символа: {a, b, c}.

1.4. СИНТАКСИЧЕСКИЕ ДЕРЕВЬЯ

Результат анализа исходного предложения в терминах грамматических конструкций удобно

представлять в виде дерева. Такие деревья принято называть синтаксическими.

На рисунке 3, а изображено дерево грамматического разбора для предложения `read(a)` с помощью правил 12 и 4 грамматики, представленной на рис. 2.

На рисунке 3, б приведено синтаксическое дерево разбора предложения `rez:=sum div100-a*a`. Это дерево выводится с помощью правил 8, 9, 10, 11 грамматики, представленной на рис. 2.

Таким образом, рассматривая правила грамматики, начиная с начального, можно построить синтаксическое дерево для всей программы, представленной на рис. 1.

2. ТЕОРИЯ ТРАНСЛЯЦИИ

План:

1. Лексический анализ.
2. Синтаксический анализ. 3. Метод рекурсивного спуска.
4. Метод операторного предшествования. 5. Внутреннее представление программы. 6, Генерация кода.

Ключевые слова: трансляция, синтаксический анализ, постфиксная запись, внутреннее представление в виде четвёрок.

2.1. ЛЕКСИЧЕСКИЙ АНАЛИЗ

На вход компилятора, а следовательно, и лексического анализатора поступает цепочка символов некоторого алфавита. Работа лексического анализатора

заключается в том, чтобы сгруппировать определённые символы в единые синтаксические объекты, называемые лексемами. Какие объекты считать лексемами, зависит от определения языка. Кроме терминальных символов (+, — / * (,)), которые сами по себе являются лексемами, в программе некоторые комбинации символов часто рассматриваются как единые объекты. Среди типичных примеров можно указать следующие:

- В некоторых языках цепочка, состоящая из одного или более пробелов, обычно рассматривается как один пробел.
- В языках программирования есть ключевые слова, такие как `begin`, `to`, `do`, `integer` и др., каждое из которых считается одним символом.
- Каждая цепочка, представляющая цифровую константу, рассматривается как один элемент текста.
- Идентификаторы, используемые имена переменных, функций, процедур, меток и т.п. также считаются лексическими единицами алгоритмического языка.

Для программы на рис. 1 будем считать лексемами терминальные символы, относящиеся к ключевым словам, знакам операций, разделителям `program`, `var`, `begin`, `integer`, `end`, `(,)`, `:=`, `+` `—`, `*` `,` `div`, `read`, `for`, `write`, `to`, `do`) Кроме того, возможны лексемы — идентификаторы и константы.

Итак, лексический анализатор должен исходный текст программы (рис. 1) представить в виде последовательности лексем. Для эффективности последующих действий каждая лексема обычно

представляется некоторым кодом фиксированной длины (например, целым числом), а не в виде строки символов переменной длины.

Для вышеприведённого примера можно составить кодировочную таблицу (табл. 1). Если распознанная лексема является ключевым словом, разделителем или знаком операции, такая схема кодирования даёт всю необходимую информацию. В случае идентификатора или константы необходимы дополнительные данные (в простейшем случае – тип и указание на адрес ячейки памяти, где они хранятся). Обычно эти данные находятся в таблицах символов и в качестве дополнительной информации для лексемы типа идентификатор или константа может служить указатель на соответствующий элемент таблицы.

Таблица 2

Таким образом, результат обработки лексическим анализатором обрабатываемой программы можно представить последовательностью лексем (табл. 2). Здесь в качестве дополнительных данных для констант используется значение самой константы, а для идентификатора — его номер в таблице символов.

2.2. СИНТАКСИЧЕСКИЙ АНАЛИЗ

Синтаксический анализ — второй этап компиляции. Во время этого этапа предложения программы распознаются как языковые конструкции используемой грамматики. Для того, чтобы выяснить, принадлежит ли

предложение языку, необходимо построить алгоритм, который для любого предложения, допустимого грамматикой, давал бы последовательность выводов этой цепочки к начальному символу грамматики. Мы можем рассматривать этот процесс как построение дерева грамматического разбора для транслируемых предложений. Различают две категории алгоритмов разбора: нисходящий (сверху вниз) и восходящий (снизу вверх). Эти термины соответствуют способу построения синтаксических деревьев. Рассмотрим для примера предложение 35 в грамматике целых чисел:

$$N \rightarrow VLB, \text{ в } \varepsilon 0/1/2/3/4/5/6/7/8/9.$$

При нисходящем разборе дерево строится от корня (начального символа) вниз к конечным узлам (рис. 4).

Восходящий разбор состоит в том, что, отправляясь от заданной цепочки, пытаются привести её к начальному символу (рис. 5).

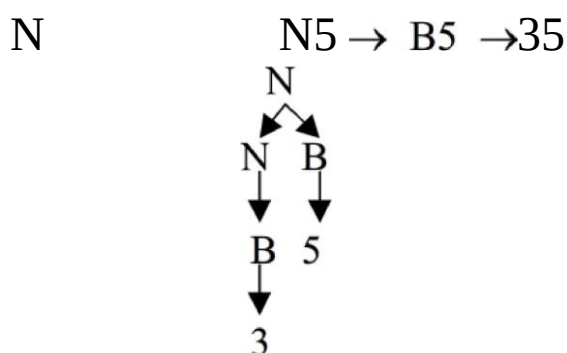


Рис. 4

$$35 \rightarrow B5$$

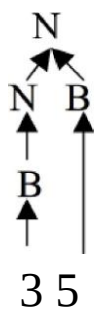


Рис. 5

Разработано множество методов синтаксического анализа. В лабораторных работах рассматриваются два метода: нисходящий и восходящий.

2.3. МЕТОД РЕКУРСИВНОГО СПУСКА

Процесс грамматического разбора для этого метода состоит из отдельных процедур для каждого нетерминального символа, определённого в грамматике. Каждая такая процедура старается во входном потоке найти подстроку, начинающуюся с текущей лексемы, которая может быть интерпретирована как нетерминальный символ, связанный с данной процедурой. В процессе своей работы она может вызывать другие процедуры или даже рекурсивно саму себя для поиска других нетерминальных символов. Если эта процедура находит соответствующий нетерминальный символ, то она заканчивает работу и передаёт в вызывающую её программу признак успешного выполнения. Затем рассматривается следующая лексема, идущая за распознанной подстрокой. Если же процедура не может найти подстроку, которая могла бы быть интерпретирована как требуемый нетерминальный символ, она заканчивается с признаком неудачи или же вызывает процедуру диагностического сообщения.

Рассмотрим в качестве примера правило грамматики:

`<ввод> read (<список переменных>)`

Процедура метода рекурсивного спуска, соответствующая нетерминальному символу `<ввод>`, прежде всего исследует две последовательные лексемы `"read"` и `"("`. В случае совпадения эта процедура вызывает другую процедуру, соответствующую нетерминальному символу `<список переменных>`. Если эта процедура закончится успешно, то процедура `<ввод>` сравнивает следующую лексему с `)"`. Если все эти проверки окажутся успешными, то процедура `<ввод>` завершается с признаком успеха и устанавливает указатель текущей лексемы на лексему, следующую за)

.

Ещё пример. Процедура, соответствующая нетерминальному символу `<оператор>`, анализирует очередную лексему для того, чтобы выбрать одну из четырёх альтернатив:

`<оператор>`

`<присваивание>/<ввод>/<вывод>/<щжл>`

Если это лексема `read`, то вызывается процедура `<ввод>`. Если это лексема, соответствующая символу идентификатор, то вызывается процедура `<присваивание>`, поскольку это единственная альтернатива, которая может начинаться с лексемы идентификатор, и т.д.

Но если мы попытаемся написать полный набор процедур для грамматики, то столкнёмся со следующей трудностью — процедура для нетерминала `<список переменных>` будет не в состоянии выбрать одну из двух

альтернатив, поскольку обе альтернативы: `ид` и `<список переменных>` могут начинаться с лексемы `ид`.

`<список переменных> —> ид / <список
переменных>, ид`

Тут скрыта и более существенная трудность. Если процедура каким-либо образом решит попробовать альтернативу `<список переменных> / ид`, то она немедленно вызовет рекурсивно саму себя для поиска нетерминального символа `<список переменных>`. Это приведёт ещё к одному рекурсивному вызову и т.д., в результате чего образуется бесконечная цепочка рекурсивных вызовов. Те же проблемы возникнут и для некоторых других правил грамматики (`<раздел переменных>`, `<раздел операторов>`, `<арифметическое выражение>`, `<слагаемое>`). Как избежать такой рекурсии? Для этого применяют другую запись грамматики. Например:

`<список переменных> ид {, ид}`

Эта запись, являющаяся широко принятым расширением БНФ, означает, что конструкция, заключённая в фигурные скобки, может быть либо опущена, либо повторяться один или более число раз. Таким образом, это правило определяет нетерминальный символ `<список переменных>` как состоящий из единственной лексемы `ид` или же из произвольного числа следующих друг за другом лексем `ид`, разделённых запятой. Это, бесспорно, эквивалентно ранее принятому правилу. В соответствии с этим новым определением процедура `<список переменных>` сначала ищет лексему `ид`, а затем продолжает сканировать входной текст до тех пор, пока следующая пара лексем

не совпадёт с запятой и ид. Такая запись устраняет проблему рекурсии, а также решает вопрос выбора из двух альтернатив.

Грамматика языка, к которому принадлежит предложение (рис. 1), имеет вид, представленный на рис. 6.

```
<программа> <имя программы> var <раздел  
переменных> begin <раздел операторов> end.  
<имя программы> ид  
<раздел переменных> <список  
переменных>:<тип>  
<список переменных> ид {, ид}  
<тип> integer  
<раздел операторов> —><оператор>  
{;<оператор>}  
<оператор>  
<присваивание>/<ввод>/<вывод>/<щжл>  
<присваивание> ид := <арифметическое  
выражение>  
<арифметическое выражение> <слагаемое>  
{+<слагаемое>} {-<слагаемое>}  
<слагаемое> —» <значение> {*<значение>}  
{div <значение>}  
<значение> ид! конст/(<арифметическое  
выражение>)  
<ввод> read (<список переменных>)  
<вывод> write (<список переменных>)  
<цикл> for <выражение цикла> to <тело цикла>  
<выражение цикла> ид :=<арифметическое  
выражение> do <арифметическое выражение>
```

```
<тело цикла> <опе ато >/be in < азел опе ато  
ов> end
```

Рис. 6

Рассмотрим примеры алгоритмов синтаксического анализа методом рекурсивного спуска для некоторых предложений исходной программы с использованием приведённой грамматики.

Имеем предложение исходной программы: READ (a).

Тогда процедура разбора этого предложения может иметь вид:

```
procedure <ввод>; begin BP false; ift read then begin  
перейти к следующей лексеме; if ( then begin  
перейти к следующей лексеме; if <список переменных>  
закончилась успешно then ift = ) then begin  
BP := Ите; перейти к  
следующей лексеме; end; {if)} end; {if }end;
```

```
{ifread} ifBP = true then успешное
завершение else неудачное завершение; end;
```

В приведённой процедуре BP — вспомогательная переменная, а t — переменная, определяющая тип лексемы. Процедура, соответствующая нетерминальному символу <ввод>, вызывает процедуру <список переменных>:

```
procedure <список
переменных >;begin
BP false;
ifthen
begin
BP true•, перейти к следующей
лексеме; while ( , ) and ( BP = true ) do
begin перейти к следующей лексеме; ift ид
then перейти к следующей лексеме else BP :=
false; end; {while} if BP = true then успешное
завершение else неудачное завершение; end;
```

На рисунке 7 графически представлен процесс грамматического разбора методом рекурсивного спуска для предложения READ. На рисунке 7, а изображен вызов процедуры <ввод>, которая обнаружила лексемы READ и) во входном потоке (штриховая линия). На рисунке 7, б процедура <ввод> вызывает процедуру <список переменных> (сплошная линия), которая обработала лексему ид. На рисунке 7, в процедура <список переменных> закончила работу, передала управление процедуре <ввод> с признаком успешного завершения; процедура <ввод> обработала входную

лексему). На этом анализ входного предложения завершён.

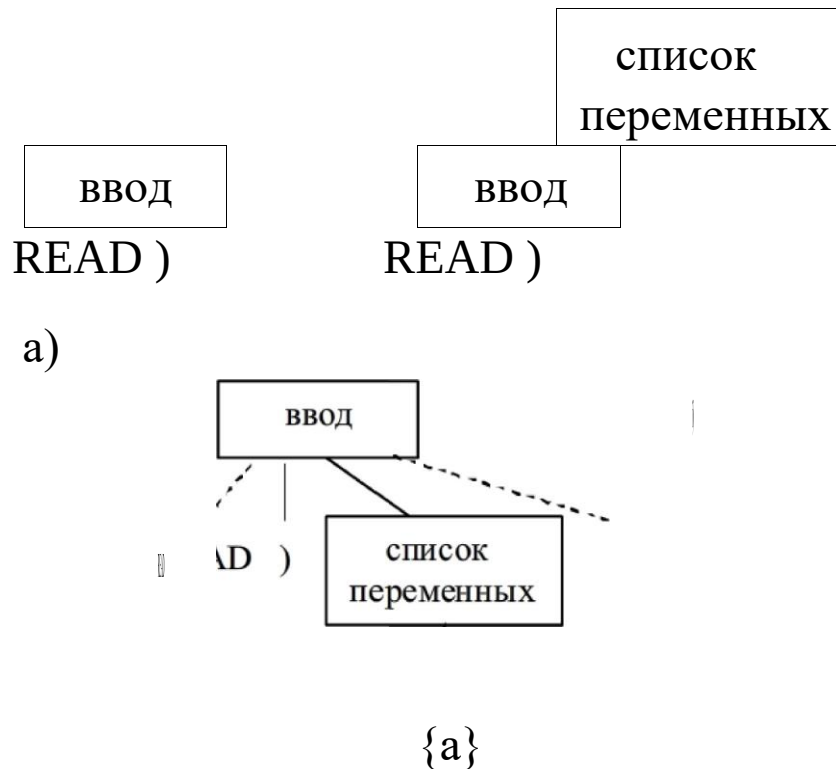


Рис. 7

Приведём ещё один пример. Имеем предложение из исходной программы:

`rez := sum div 100 — A * A.`

Представим алгоритмы разбора этого предложения методом рекурсивного спуска:

```

procedure <присваивание> begin BP false•, if t =
    ид then begin перейти к следующей лексеме;
    if t then begin перейти к следующей лексеме;
    if <арифметическое выражение>
        завершилось успешно then
            BP true•, end; {if
:=}end; {if ид.if BP = true then

```


успешное завершение else
неудачное завершение; end;

Процедура присваивание в процессе работы
вызывает процедуру <арифметическое выражение>:

```
procedure          арифметическое
  выражение; begin ВР:=false; if
  <слагаемое>          завершилось
  успешно then begin
    ВР:=true; while (t = + или t = - )
    and ( ВР=true ) do begin
      Перейти к следующей лексеме; if
      <слагаемое>          завершилось
      неудачно then
        ВР:=false;
    end; {while} end; {if
слагаемое} if ВР true then
успешное завершение else
неудачное завершение; end;
```

Процедура <арифметическое выражение> в
соответствии с грамматикой вызывает процедуру
<слагаемое>: procedure <слагаемое> ; begin

```
  ВР:=false; if <значение>
  завершилось успешно then begin
    ВР:=true; while (t * или t div ) and
    ( ВР=true ) do begin
      Перейти к следующей лексеме;
      if <значение> завершилось
      неудачно then
        ВР:=false;
    end; {while} end; {if
значение} if ВР=true then
```

успешное завершение else
неудачное завершение; end;

И, наконец, процедура <слагаемое> вызывает процедуру <значение>, которая распознает переменные, константы или вызывает процедуру <арифметическое выражение>. Алгоритм процедуры <значение> имеет вид:

```
procedure значение;  
begin  
  ВР:=false;  
  ift= ид или конст then begin  
    ВР:=true;  
    Перейти к следующей  
    лексеме; end или конст}  
  else ift=  
  ( then  
  begin  
    Перейти к следующей лексеме; if  
    <арифметическое выражение>  
    завершилось успешно then if t=) then  
    begin  
      ВР:=true;  
      Перейти к следующей  
      лексеме; end; {if)} end; {if()} if ВР п-ие  
  then успешное завершение else  
  неудачное завершение; end;
```

Графически разбор этого предложения методом рекурсивного спуска выглядит следующим образом:

1. Вызывается процедура <присваивание>, которая обнаружила лексемы ид и во входном потоке (рис. 8, а).

2. Процедура <присваивание> вызывает процедуру <арифметическое выражение> (рис. 8, б).

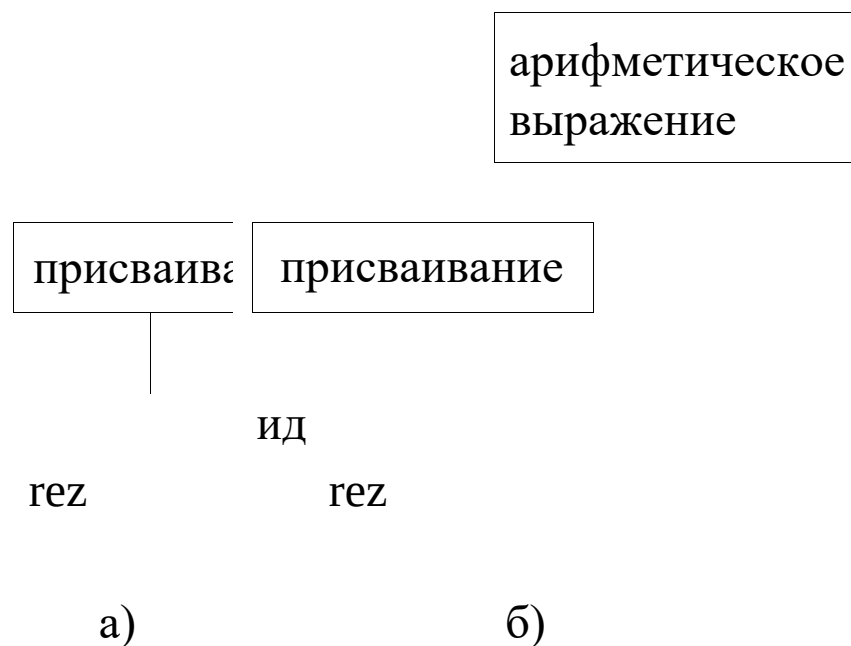
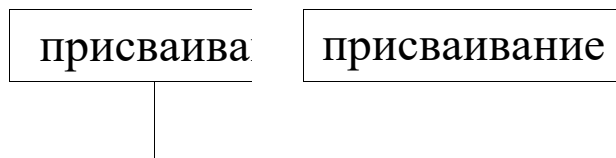


Рис. 8



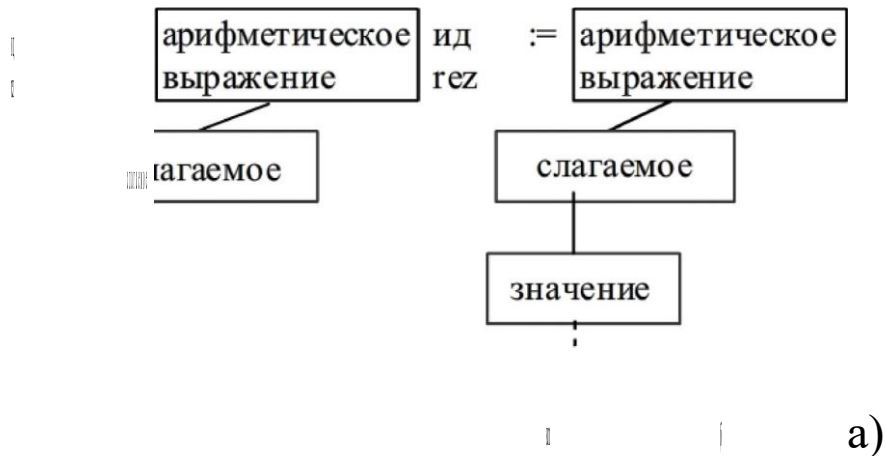


Рис. 9

3. Процедура <арифметическое выражение> вызывает процедуру <слагаемое> (рис. 9, а).

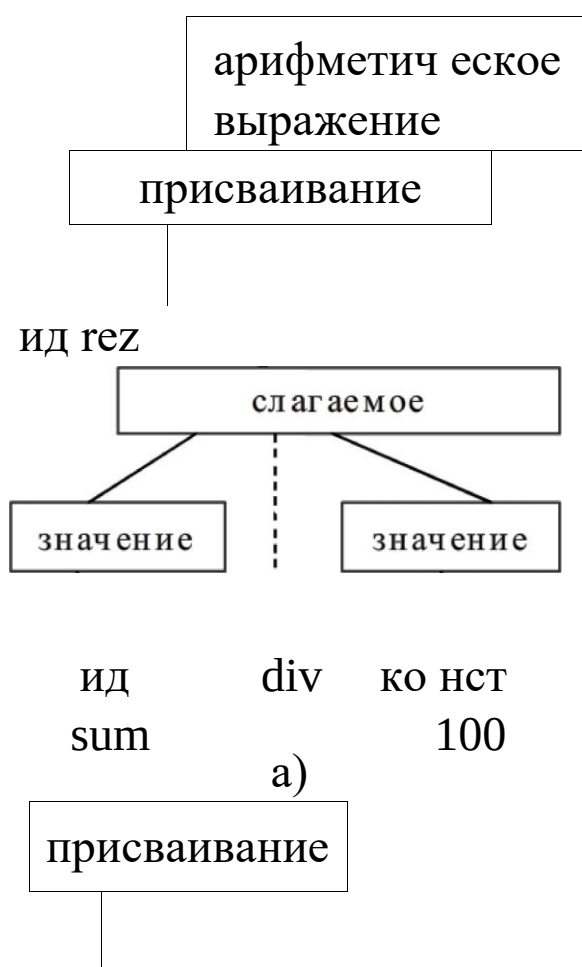
4. Процедура <слагаемое>. вызывает процедуру <значение>, которая обнаруживает лексему ид. Управление возвращается в процедуру <слагаемое> (рис. 9, б).

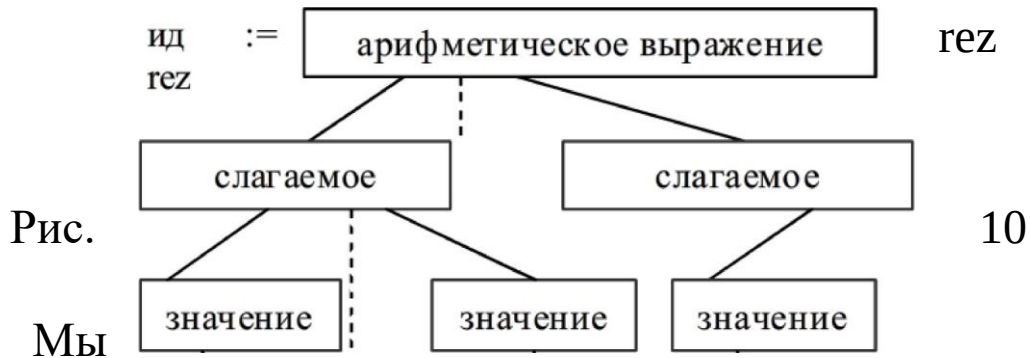
5. Процедура <слагаемое> обнаруживает лексему div и вызывает процедуру <значение>, которая обнаруживает лексему конст и передаёт управление в процедуру <слагаемое>, которая передаёт управление в процедуру <арифметическое выражение> (рис. 10, а).

6. Процедура <арифметическое выражение> распознаёт лексему -, затем вызывает процедуру <слагаемое>, которая вызывает процедуру <значение>, которая распознаёт лексему ид и передаёт управление в процедуру <слагаемое> (рис. 10, б).

7. Процедура <слагаемое> распознаёт лексему *, затем вызывает процедуру <значение>, которая распознаёт лексему ид. Следующая лексема читается в процедуре <значение>. Эта лексема не относится к данному предложению. Управление передаётся в

процедуру <слагаемое>, которая формирует признак успешного завершения и передаёт управление в процедуру <арифметическое выражение>. Эта процедура, в свою очередь, успешно заканчивается и передаёт управление в процедуру <присваивание>, которая формирует признак успешного завершения. На этом разбор предложения заканчивается (рис. 11).





привели примеры грамматического разбора отдельных предложений методом рекурсивного спуска. Однако этот метод применим и ко всей программе в целом. В этом случае для осуществления синтаксического анализа следует просто обратиться к процедуре, соответствующей нетерминальному символу <программа>.

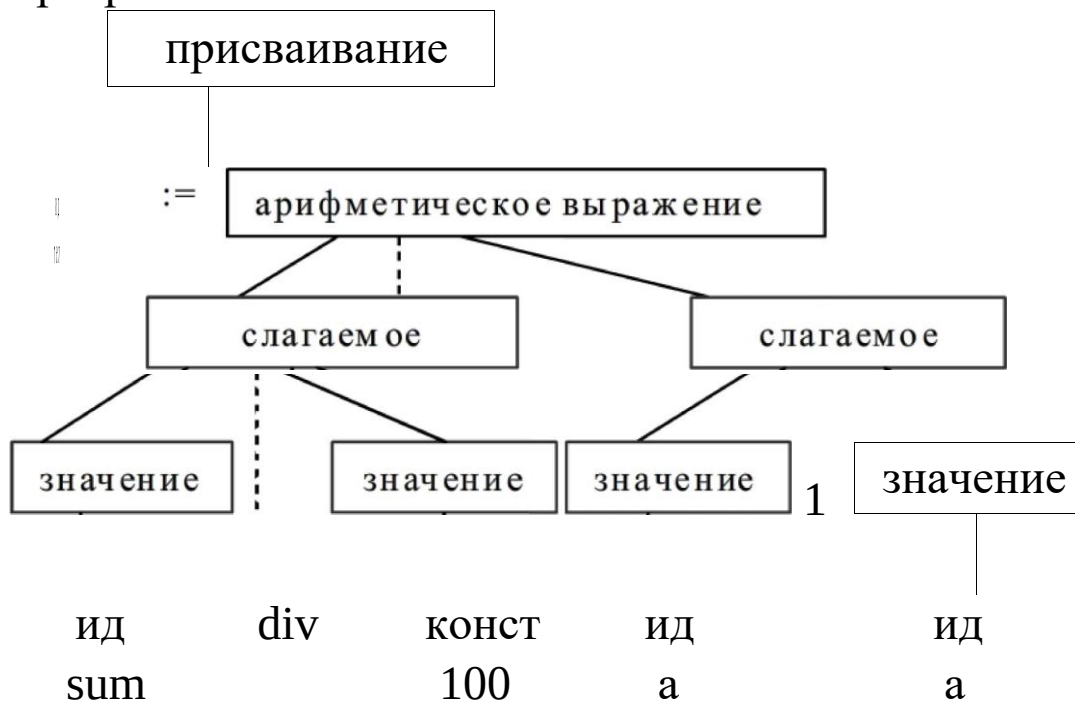


Рис. 11

В результате работы этой процедуры будет построено дерево грамматического разбора для всей программы.

2.4. МЕТОД ОПЕРАТОРНОГО ПРЕДШЕСТВОВАНИЯ

Этот метод относится к восходящим (метод снизу вверх), которые начинают разбор с конечных узлов грамматического дерева и пытаются объединить их построением узлов всё более и более высокого уровня до тех пор, пока не будет достигнут корень дерева. Метод операторного предшествования основан на анализе пар последовательно расположенных операторов исходной программы и решением вопроса о том, какой из них должен выполняться первым. Рассмотрим, например, арифметическое выражение

$$A+B*C-V.$$

В соответствии с обычными правилами арифметики умножение и деление осуществляются до сложения и вычитания. Можно сказать, что умножение и деление имеют более высокий уровень предшествования, чем сложение и вычитание. При анализе первых двух операторов (+,*) выяснится, что оператор + имеет более низкий уровень предшествования, чем оператор *. Часто это записывают следующим образом:

Аналогично для следующей пары операторов (* и —) оператор * имеет более высокий уровень предшествования, чем оператор —. Мы можем записать это в виде

Метод операторного предшествования использует подобные отношения между операторами для управления процессом грамматического разбора. В частности, для рассмотренного арифметического

выражения мы получили следующие отношения предшествования:

$$A + B * C - B$$

Отсюда следует, что подвыражение $B * C$ должно быть вычислено до обработки любых других операторов рассматриваемого выражения. В терминах дерева грамматического разбора это означает, что операция $*$ расположена на более низком уровне узлов дерева, чем операция $+$ или $-$. Таким образом, рассматриваемый метод грамматического разбора должен распознать конструкцию $B * C$, интерпретируя её в терминах заданной грамматики, до анализа соседних термов предложения.

Предшествующее изложение иллюстрирует основную идею, на которой основан метод грамматического разбора, построенный на отношениях операторного предшествования. В рамках этого метода предложение сканируется слева направо до тех пор, пока не будет найдено подвыражение, операторы которого имеют более высокий уровень предшествования, чем соседние операторы. Далее это подвыражение распознаётся в терминах правил вывода используемой грамматики. Этот процесс продолжается до тех пор, пока не будет достигнут корень дерева, что и будет означать окончание процесса грамматического разбора. Далее мы рассмотрим приложение описанного подхода к нашему примеру программы (рис. 1). Грамматика этого предложения имеет вид, представленный на рис. 2.

Первым шагом при разработке процессора грамматического разбора, основанного на методе операторного предшествования, должно быть установление отношений предшествования между операторами грамматиками. При этом под оператором понимается любой терминальный символ (т.е. любая лексема). Таким образом, мы должны, в частности, установить отношения предшествования между лексемами `begin`, `read`, `(`. Приведём матрицу, которая задаёт отношения предшествования для нашей грамматики (табл. 4).

Таблица 4

Лексемы	var	begin	end	integer	for	read	write	to	do	;	:	,	:=	+	-	*	div	(	)	ид	конст
program																				✓	
var												✓								✓	
begin							✓													✓	
end				✓																	
integer		✓																			
for																				✓	
read																					
write																					
to									✓												
do		✓					✓													✓	
;		✓					✓													✓	
:																					
,																					
:=																				✓	
+																				✓	
-																				✓	
*																				✓	
div																				✓	
(																				✓	
)																				✓	
ид																					
конст																					

Каждая клетка этой матрицы определяет отношение предшествования (если оно существует) между лексемами, соответствующими строке и столбцу, на пересечении которых находится эта клетка. Например, мы видим, что

```
program var и  
begin <• read
```

Отношение = означает, что обе лексемы имеют одинаковый уровень предшествования и должны рассматриваться грамматическим процессором в качестве составляющих одной конструкции языка. Обратите внимание, что для отношения предшествования не выполняются некоторые правила, привычные для отношения арифметического порядка. Например,

```
;•> end, но end •>;
```

Обратите внимание также на то, что для многих пар лексем отношения предшествования не существует. Это означает, что соответствующие пары лексем не могут находиться рядом ни в каком грамматически правильном предложении. Если подобная комбинация лексем всё же встретится в процессе грамматического разбора, то она должна рассматриваться как синтаксическая ошибка.

Для применимости метода операторного предшествования необходимо, чтобы отношения предшествования были заданы однозначно, например, не должно быть одновременно отношений ; <• begin и ; >• begin. Это требование выполняется для нашей грамматики, однако, несущественные её изменения

могут привести к тому, что некоторые из отношений перестанут быть однозначными и метод операторного предшествования станет не применимым.

Приведём пример разбора с помощью операторного предшествования. Пусть анализируется предложение `read` из нашей программы. Это предложение анализируется по лексемам слева направо. Для каждой пары соседних операторов определено отношение предшествования `begin read ( ид ) ; begin read ( <N1> ) ;`
`begin <N`

На первом шаге процессор грамматического разбора выделим фрагмент, ограниченный отношениями `<•` и `•>` для распознавания в терминах грамматики. В данном случае этот фрагмент содержит единственную лексему `ид`. Эта лексема может быть распознана как нетерминал `<значение>` в соответствии с правилом из грамматики. Однако эта лексема может быть также распознана как нетерминальный символ `<список переменных>`. Для рассматриваемого метода не важно, какой конкретно нетерминальный символ распознан. Лексема `ид` интерпретируется просто как некий нетерминальный символ `<N1>`. Конструкция `read(<N1>)` интерпретируется как один нетерминальный символ `<N2>`.

На этом разбор предложения `READ` закончено. Если мы сравним деревья грамматического разбора для этого предложения, то увидим, что они полностью совпадают, за исключением имён нетерминальных символов.

Рассмотрим грамматический разбор для

оператора `rez.:=sum div 100 - A * A;`.

= < > < > < > < >
`rez := <N1> div <N2> - <N4> *. rez :=. rez:=`

При этом дерево грамматического разбора имеет вид, представленный на рис. 12.

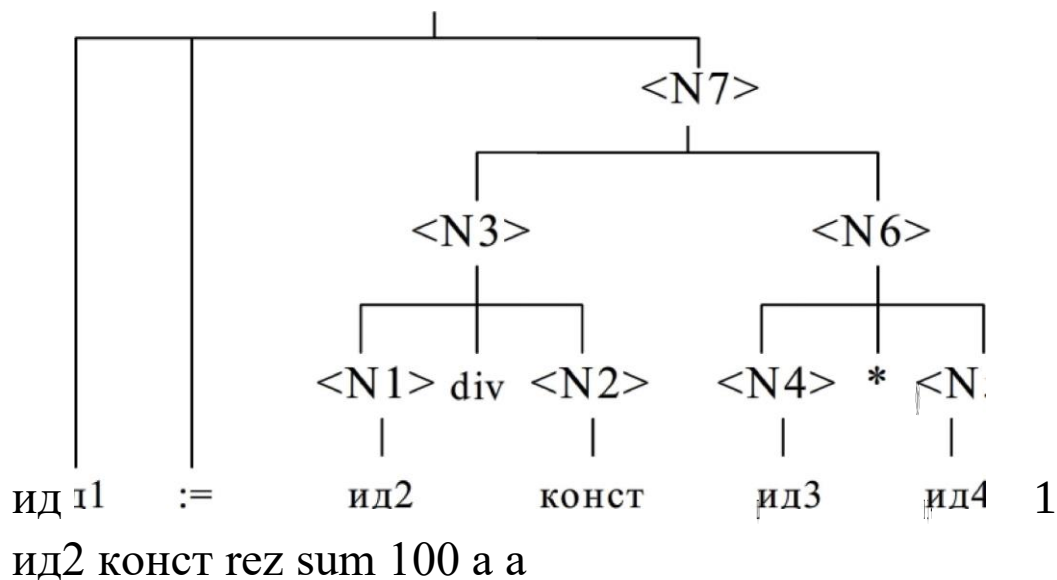


Рис. 12

Заметим ещё раз, что процесс грамматического разбора начинается слева направо и продолжается на каждом шаге до тех пор, пока не определится очередной фрагмент предложения для грамматического распознавания, т.е. первый фрагмент, ограниченный отношениями `<•` и `•>`. Как только подобный фрагмент выделен, он интерпретируется как некоторый очередной нетерминальный символ в соответствии с какимнибудь правилом грамматики. Этот процесс продолжается до

тех пор, пока предложение не будет распознано целиком.

Обратите внимание, что каждый фрагмент дерева грамматического разбора строится, начиная с конечных узлов, вверх, в сторону корня дерева. Отсюда и возник термин восходящий разбор. Если мы рассмотрим дерево грамматического разбора, исходя из грамматики языка (рис. 2), то увидим, что оно несколько отличается от полученного методом операторного предшествования дерева (рис. 12).

Например, идентификатор `sum` был сначала интерпретирован как `<значение>`, а потом как `<слагаемое>`, являющийся одним из операндов операции `div`. При разборе методом операторного предшествования идентификатор `sum` был интерпретирован как единственный нетерминал `<N1>`, который является операндом операции `div`. Таким образом, `<N1>` соответствует двум нетерминальным символам `<значение>` и `<слагаемое>`. Имеются и другие подобные различия.

Они вытекают из свободы образования имен нетерминальных символов, распознаваемых в рамках метода операторного предшествования. Интерпретация `sum` сначала как `<значение>`, а потом как `<слагаемое>` является просто переименованием нетерминальных символов. Такое переименование необходимо, поскольку в соответствии с грамматикой (правило 8) первым операндом операции умножения должен быть `<слагаемое>`, а не `<значение>`, так как для нашего метода имена нетерминальных символов несущественны, то подобные переименования

становятся ненужными. Собственно говоря, три различных имени: <арифметическое выражение>, <слагаемое>, <значение> — были включены в грамматику только как средства описания отношения предшествования между операторами (например, для указания того, что умножение следует выполнять после сложения). Поскольку эта информация содержится в нашей матрице предшествования, то становится ненужным различать эти три имени в процессе грамматического разбора.

Возможный алгоритм метода операторного предшествования приведён на рис. 13.

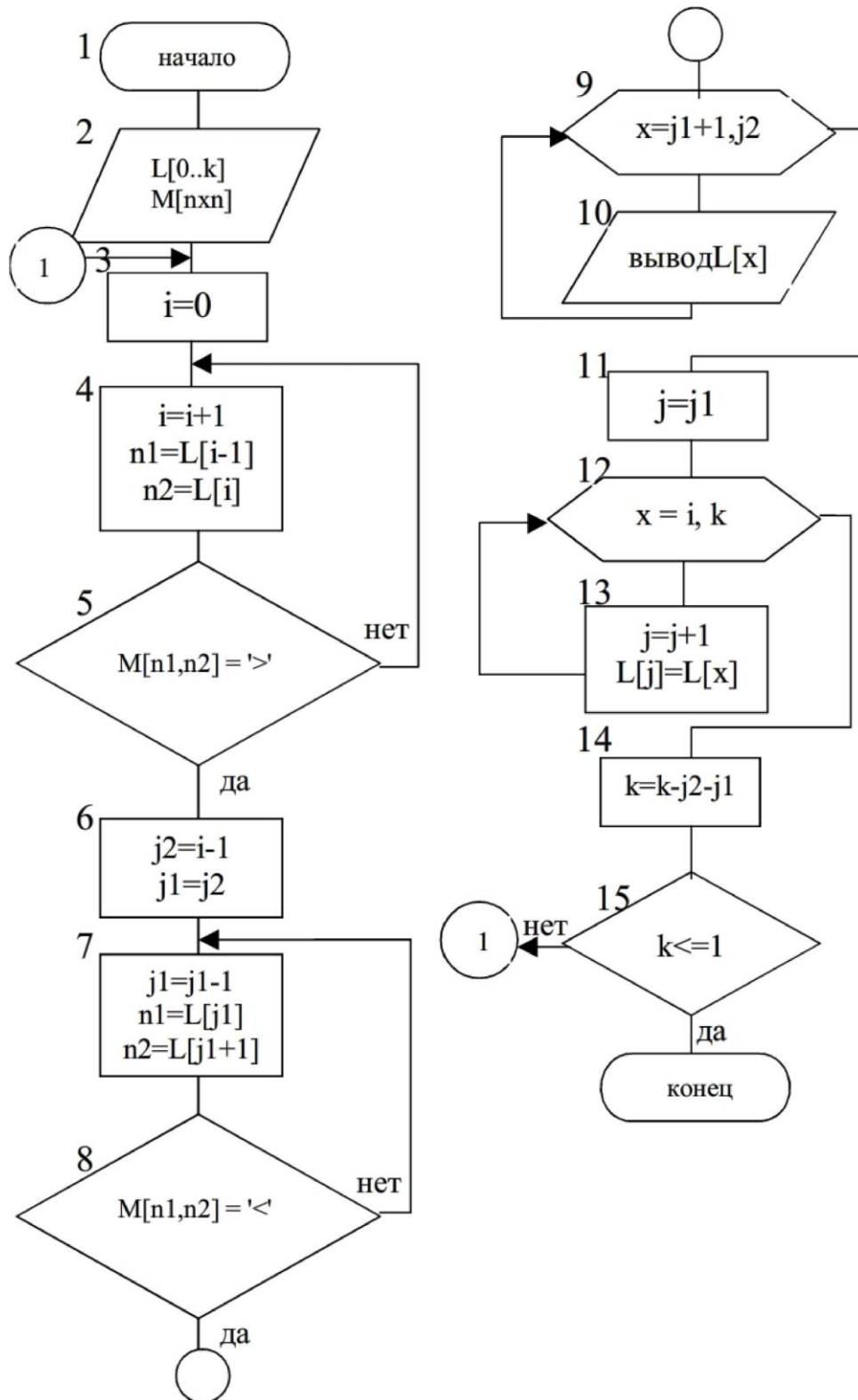


Рис. 13

В данном алгоритме первоначально просматривается цепочка лексем (массив L), устанавливается отношение предшествования между соседними лексемами по таблице отношений предшествования (матрица M[пхп]) до тех пор, пока не встретится отношение '>' (блоки 4 — 5). После этого возвращаемся по массиву лексем назад, пока между лексемами не встретится отношение (блоки 7 — 8). Затем лексемы, ограниченные отношениями К Я, записываются во внутреннее представление (блоки 9 — 10) . В блоках 12 — 14 элементы массива лексем сдвигаются на длину выведенной цепочки. Так продолжается, пока не распознана вся последовательность лексем.

2.5. ВНУТРЕННЕЕ ПРЕДСТАВЛЕНИЕ ПРОГРАММЫ

На выходе синтаксического анализатора формируется программа во внутреннем представлении. Существует несколько различных способов представления программы в некоторой промежуточной форме для анализа и оптимизации кода: последовательность четвёрок, последовательность троек, постфиксная запись, префиксная запись, синтаксическое дерево.

1. Последовательность четвёрок

Каждая четвёрка записывается в виде:

операция op1, op2, результат, где операция — выполняемая объектным кодом функция; op1, op2 – операнды этой операции; результат

определяет, куда должно быть помещено результирующее значение.

Например, предложение исходной программы (рис. 1): `sum:=sum+a` может быть представлено четвёрками следующим образом:

`sum, a ,sum`

Здесь 11 обозначает промежуточный результат (`sum+a`), вторая четвёрка присваивает это значение переменной `sum`.

Все четвёрки расположены в том порядке, в котором должны выполняться соответствующие инструкции объектного кода, что существенно облегчает анализ для оптимизации кода. Это означает также, что трансляция в машинные коды будет относительно простой. В таблице 3 представлена последовательность четвёрок, соответствующая исходной программе.

За операциями и `write` следует четвёрка `Param`, определяющая параметры операций `read` и `write`. Четвёрка `Param` будет, разумеется, при окончательной генерации машинного кода оттранслирована в список параметров. Операция `>` в четвёрке 3 сравнивает значение двух своих операндов и осуществляет переход к четвёрке 9, если первый

операнд больше второго. Операция goto в четвёрке 8 осуществляет безусловный переход к четвёрке 3.

Таким образом, последовательность четвёрок и является результатом работы синтаксического анализатора.

2. Постфиксная запись

Обычно программа осуществляет те или иные действия над данными. Соответствующие операции программист записывает, используя инфиксную форму записи, в которой знак операции ставится между операндами. Например: $(A + B) * C$.

Вычисление такого выражения является непростой задачей. Операцию умножения нельзя выполнить до тех пор, пока не будет прочитан второй операнд C. Если этот операнд сам является сложным выражением, то прежде чем выполнить умножение, необходимо считать много данных из текста программы.

Отмеченные трудности можно легко обойти, если использовать другую форму записи операций. Она называется постфиксной и отличается тем, что знак операции ставится непосредственно за операндами. Такая запись обладает двумя ценными свойствами, благодаря которым её используют как промежуточную форму представления исходной программы при трансляции:

30

1. Для записи любого выражения не нужны скобки. Так как оператор непосредственно следует за операндами, участвующими в операции, неопределённость в указании операндов отсутствует.

Например, выражение $(A + B) * C$ в постфиксной записи имеет вид:

$$AB+C*.$$

2. К моменту считывания очередного оператора соответствующие операнды уже прочитаны. Поэтому оператор может быть выполнен без чтения каких либо дополнительных данных.

Сказанное выше относится к бинарным операциям, однако не трудно распространить результаты рассуждений и на унарные операции. Однако при этом могут возникнуть сложности. Например, знак "11" может стоять в инфиксной записи, указывая как бинарную, так и унарную операцию, и его правильный смысл становится очевидным из контекста. В постфиксной записи сделать это труднее. Унарный минус и другие унарные операции можно представлять двумя способами: либо записывать их как бинарные операции, например вместо "— В" писать "0 — В"; либо для обозначения унарных операций вводить новый символ, например, выражение $A + (- B + C * E)$ в постфиксной форме примет вид: $A B @ C E * + + .$

2.6. ГЕНЕРАЦИЯ КОДА

Возможны три формы объектного кода: абсолютные команды, помещённые в фиксированные ячейки (после окончания компиляции такая программа немедленно выполняется); программа на автокоде (ее потом придётся транслировать); программа на языке машины, записанная на внешнюю память (для выполнения она должна быть объединена с другими подпрограммами и затем загружена).

Первый вариант наиболее экономичен в отношении расходуемого времени. Главный недостаток этого варианта состоит в том, что нельзя предварительно и независимо протранслировать несколько подпрограмм и затем объединить их вместе для выполнения, все подпрограммы должны транслироваться одновременно. Выигрыш во времени оборачивается проигрышем в гибкости. Проще всего получить объектную программу на автокоде. В этом случае не приходится формировать команды как последовательности битов; можно порождать команды, содержащие символические имена. Более того, можно формировать макроопределение. Это позволяет также уменьшить объём компилятора. Несмотря на очевидные достоинства, трансляция на автокод обычно считается наихудшим из вариантов. И в самом деле, к процессу трансляции добавляется ещё один шаг, который часто требует столько же времени, сколько длится собственно компиляция. Большинство промышленных компиляторов вырабатывают объектную программу в виде объектного модуля. Как правило, объектный модуль содержит символические имена других программ (подпрограмм), к которым он обращается, и имена своих входных точек, к которым можно обращаться из других программ. Эта объектная программа “объединяется” с теми другими объектными программами, а затем загружается в некоторую область памяти для выполнения.

В этом варианте обеспечивается гораздо большая гибкость, и поэтому во многих системах он и принят в качестве стандартной процедуры. Следует, однако,

заметить, что на объединение и загрузку таюке расходуется время.

Теперь покажем, как генерируются команды для последовательности четвёрок и постфиксной записи, используя в качестве примера выражение

$$A * ((A * B + C) - C * B).$$

Будем считать переменные A, B, C, B целыми.

Генерация кода для последовательности четвёрок.

Для рассматриваемого примера последовательность четвёрок имеет вид:

-4- C 12

_ п 13 14

* a 14 15

В основе процедуры генерации кода лежит оператор case:

cedure ГК; case код операции
четвёрки of : подпрограмма,
соответствующая операции;

+ : подпрограмма, соответствующая
операции + ; - : подпрограмма,
соответствующая операции - ;end;

Генерация кода для постфиксной записи. Для рассматриваемого примера постфиксная запись имеет вид:

$$A A B * C + C D * - A *.$$

Операторы и операнды просматриваются последовательно слева направо. Всякий раз, когда просматривается операнд, в стек заносится его имя, а

когда встречается операция, генерируются команды для ее выполнения. При этом в качестве описаний операндов используются два верхних описания в стеке; затем эти два описания заменяются описанием результата. При этом необходимо сформировать временное имя T_i , которое заносится в стек. На практике стек можно отобразить на одномерный массив $S(1), S(2) \dots S(n)$. Для указания вершины стека можно использовать индекс i . При записи в стек указатель вершины будет сдвигаться в сторону конца массива, при чтении из стека указатель вершины будет перемещаться в сторону начала массива (рис. 14).

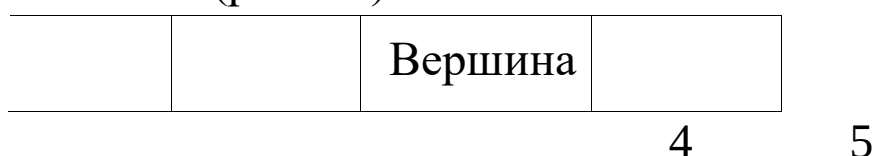


Рис. 14

Для обработки доступен только элемент $S(i)$, т.е. вершина стека. Значение $i = 0$ перед чтением из стека служит признаком того, что стек пуст, а значение $i = n$ перед записью в стек — признаком того, что стек переполнен.

3. ОРГАНИЗАЦИЯ ТАБЛИЦ СИМВОЛОВ

План:

1. Способы организации таблиц символов.
2. Неупорядоченные и упорядоченные таблицы.
3. Хеш-адресация.
4. Рехеширование.
5. Метод цепочек.

Ключевые слова: таблица символов, поиск, коллизия.

3.1. СПОСОБЫ ОРГАНИЗАЦИИ ТАБЛИЦ СИМВОЛОВ

Проверка правильности семантики и генерация кода требуют знания характеристик идентификаторов, используемых в программе на исходном языке. Эти характеристики выясняются из описаний и из того, как идентификаторы используются в программе. Вся информация накапливается в таблицах символов.

Таблицы всех типов имеют общий вид (табл. 6). В нашем случае аргументами таблицы являются символы или идентификаторы, а значениями — их характеристики. Когда компилятор начинает трансляцию исходной программы, таблица символов пуста или содержит только несколько элементов для служебных слов и стандартных функций. В процессе компиляции для каждого нового идентификатора элемент добавляется только один раз, но поиск ведётся всякий раз, когда встречается этот идентификатор. Так как на этот процесс уходит много времени, важно выбрать такую организацию таблиц, которая допускала бы эффективный поиск.

Таблица 6

Таблица 7

Простейший способ организации таблицы состоит в том, чтобы добавлять элементы в порядке их поступления без каких-либо попыток упорядочения. Поиск в этом случае требует сравнения с каждым элементом таблицы, пока не будет найден подходящий. Для таблицы, содержащей N элементов, в среднем будет

выполнено $N/2$ сравнений. Если N велико, такой способ неэффективен. Поиск может быть выполнен более эффективно, если элементы таблицы упорядочены согласно некоторому естественному порядку аргументов. В нашем случае, когда аргументами являются строки символов, наиболее естественным является упорядочение по алфавиту. Эффективным методом поиска в упорядоченном списке является так называемый бинарный поиск. Сортировка таблицы производится методом упорядоченных вставок.

Наиболее эффективный и широко применяемый в компиляторах метод при работе с таблицами символов — хеш-адресация. Механизм расстановки состоит из таблицы и хеш-функции (табл. 7). Таблица состоит из N элементов, где N заранее фиксировано. Метод хешадресации заключается в преобразовании символа в индекс элемента в таблице. Индекс получается хешированием символа, т.е. выполнением над ним некоторых операций. Если в процессе компиляции встретился объект a , то для поиска его в таблице можно воспользоваться следующим алгоритмом: если объект уже встречался ранее, то $h(a)$ — ячейка в таблице, в которой хранится a . Если объект a ранее не встречался, то $h(a)$ — пустая ячейка, в которую заносится информация для a .

Возникает, однако, затруднение, если результаты хеширования двух разных символов совпадают. Это называется коллизией. Очевидно, в данной позиции таблицы может быть помещён только один из этих символов, так что мы должны найти свободное место для второго. Желательно иметь такую хеш-функцию,

которая распределяла бы объекты равномерно по всей таблице, так чтобы коллизии не возникали слишком часто. Но избежать их совсем практически не удаётся, поэтому разработчику компилятора следует предусмотреть способы решения задачи коллизии. Существуют два таких способа – рехеширование и метод цепочек.

3.2. МЕТОД ЦЕПОЧЕК

Метод цепочек использует хеш-таблицу, элементы которой первоначально равны 0, собственно таблицу символов, вначале пустую, и указатель УК, который указывает на текущее положение последнего элемента в таблице символов. Элементы таблицы символов имеют дополнительное поле CHAIN, которое может содержать 0 или адрес другого элемента таблицы символов. Начальное состояние таблицы приведено на рис. 15.

Хеш-функция, применённая к символу, даёт индекс указателя в хеш-таблице. Указатель либо равен 0, либо указывает на первый элемент таблицы символов с данным значением хеш-функции. Поле CHAIN каждого элемента используется для того, чтобы связать в цепочку элементы, для которых хеширование символа приводит к тому же самому указателю. Например, в таблицу необходимо записать символ S1. Функция хеширования вырабатывает адрес элемента хештаблицы, например 4. Содержимое этой ячейки равно 0. Тогда выполняется следующее:

1. Прибавляем 1 к УК.
2. Вносим элемент (S1, значение, 0) в позицию таблицы символов, на которую указывает УК.

3. Заносим содержимое УК. в указатель 4 хеш-таблицы.

4. Пока поступают символы, хеширование которых даёт индексы разных указателей, они заносятся в таблицу аналогичным образом. Так, если мы записываем в таблицу символы S2, S3, S4, хеширование которых даёт ссылки на указатели 1, 3, 6, таблица примет вид, представленный на рис. 16.

В конце концов, поступит символ S5, который ссылается на указатель, использовавшийся ранее, например на 6. Вот здесь и начинает действовать поле CHAIN. Символ S5 записывается в таблицу символов и добавляется к концу цепочки для этого указателя (рис. 17).

Внесем в таблицу символы S6, S7, S8, которые ссылаются на указатели 4, 3, 3 соответственно (рис. 18).

18

4. ОПТИМИЗАЦИЯ КОДА

План:

1. Машинно-зависимая оптимизации.
2. Машинно-независимая оптимизация.
3. Методы оптимизации кода.

Ключевые слова: общие подвыражения, инварианты цикла.

Рассмотрим некоторые методы машинно-независимой оптимизации кода. Мы не будем стремиться к детальному описанию какоголибо из этих

методов. Вместо этого мы дадим словесное описание и проиллюстрируем основные понятия примерами. Алгоритмы и детали, касающиеся этих методов, можно найти в работах [2, 3].

Одним из важных источников оптимизации кода является удаление общих подвыражений, которые встречаются в нескольких местах программы и вычисляют одно и то же выражение. Рассмотрим, например, предложение:

```
VARx,y: ARRAY [1..10,1..10] OF INTEGER•,
```

```
FORi := 1 TO 10 DO
```

```
  x[i, 2*j-1] := y[i, 2*j];
```

Выражение $2*j$ является общим подвыражением. Оптимизирующий компилятор должен только один раз сгенерировать код, вычисляющий это умножение, и использовать его результат в обоих местах.

Общие подвыражения обычно обнаруживаются при анализе промежуточной формы представления программы (рис. 19). Следует отметить, что в первоначальном варианте требуется выполнить 161 операцию.

Если мы исследуем эту последовательность четвёрок, то обнаружим, что четвёрки 5 и 11 совпадают, за исключением имени получаемого промежуточного результата. Обратите внимание, что операнд j не меняет своего значения между четвёрками 5 и 11.

Невозможно достичь четвёрки 11, не проходя предварительно четвёрку 5, поскольку они расположены на одном линейном участке.

Таким образом, четвёрки 5 и 11 вычисляют одно и тоже значение. Это означает, что мы можем удалить четвёрку 11 и заменить любые обращения к её результату (i9) на обращение к переменной i3, которая является результатом четвёрки 5. Эта модификация позволяет избежать дублирования вычислений подвыражения $2*j$, которое мы выделили как общее подвыражение при анализе исходной программы.

После замены i3 на i10 мы обнаружим, что четвёрки 6 и 12 также совпадают, за исключением имени результата. Следовательно, мы можем удалить четвёрку 12 и заменить переменную i11 всюду, где она используется на переменную i4. Аналогично четвёрки 10 и 11 также могут быть удалены, поскольку они эквивалентны четвёркам 3 и 4. В результате получим новую последовательность четвёрок (рис. 20), которая предполагает выполнение всего 121 операции.

Обратите внимание, что общее количество четвёрок сокращено с 17 до 13. Поскольку операции во всех используемых здесь четвёрках займут, вероятно, примерно одинаковое время на обычном компьютере, то мы также сократим общее время выполнения программы.

Другим источником оптимизации кода является удаление инвариантов цикла. Так называется подвыражение внутри цикла, результирующие значения которых не изменяются внутри цикла при переходе от

одной итерации к другой. Таким образом, эти значения могут быть

вычислены только один раз перед входом в тело цикла вместо того, чтобы вычислять их заново перед каждой итерацией. Поскольку для большинства программ основное время работы приходится на выполнение циклов, экономия времени от подобной оптимизации может быть весьма существенной.

Примечание: необходимо выполнить 94 операции.

Примером инварианта цикла является вычисление выражения $2*j$. Результат вычисления этого выражения зависит только от операнда j , значение которого не изменяется во время выполнения цикла. Таким образом, мы можем поместить четвёрку 5 непосредственно перед началом выполнения цикла. Аналогичные соображения относительно четвёрок 6 и 7.

В результате получим новую последовательность четвёрок (рис. 21). Общее количество четвёрок остаётся тем же, но количество четвёрок в цикле уменьшилось с 12 до 9. Каждое выполнение предложения FOR вызывает десятикратное выполнение тела цикла. Это означает, что общее количество операций, необходимых для выполнения FOR, сократилось со 121 до 94.

Общее количество операций, приходящихся на одно выполнение предложения FOR, по сравнению с начальным вариантом сократилось со 161 до 94, что существенно уменьшило выполнение программы.

Существуют также и более тонкие методы обработки общих подвыражений и инвариантов цикла, чем описанные выше. Можно ожидать, что благодаря

этим методам может быть получен ещё более оптимизированный вариант кода.

21

5. ОРГАНИЗАЦИЯ ДИАЛОГА В ВЫЧИСЛИТЕЛЬНЫХ СИСТЕМАХ

План:

1. Этапы проектирования.
2. Типы диалога.
3. Проектирование диалога типа «меню».
4. Проектирование диалога типа «вопрос—ответ».
5. Проектирование диалога типа «заполнение бланка».
6. Проектирование диалога типа «прямой режим».
7. Средства помощи и поддержки в интерактивной системе.

Ключевые слова: диалоговые вычислительные системы, пользователь, информационная система.

Важнейшим условием правильной эксплуатации вычислительной системы является продуманная организация взаимодействия пользователя и комплекса технических средств.

Часто компьютерную систему используют непрофессиональные пользователи, поэтому некоторые важные их характеристики требуют пристального внимания. По одному из определений, непрофессиональными считаются пользователи, которые обращаются к компьютерным системам в своей повседневной работе или жизни эпизодически. Система должна соответствовать потребностям всех потенциальных пользователей. Существует разница

между потребностями тех, кто может случайно воспользоваться системой, и тех, кто может одной системой пользоваться часто, а другой только изредка. Необходимо удовлетворить потребности и пользователя, решающего определённую задачу с помощью интерактивной системы, и научного работника, ис-

пользующего программы, которые он сам написал, и администратора, который использует информацию из базы данных, и бухгалтера, желающего рассчитать экономические показатели, и ещё многих других пользователей.

Часто, из-за стремления ускорить процесс разработки, разработанные системы являются неудобными в работе и не пользуются успехом у пользователей. Удачное внедрение интерактивной системы определяется несколькими факторами:

1. Прежде всего важна окупаемость затраченных средств.
2. В разработанном пакете программ может не оказаться возможностей, которые необходимы для данного конкретного приложения или, наоборот, пользователи могут оказаться недостаточно подготовленными, чтобы в полной мере воспользоваться всеми возможностями системы.
3. Важной является скорость выполнения поставленных перед системой задач.
4. Должна быть высокой наглядность представления данных.
5. Существенно, чтобы новая интерактивная система не вносила путаницы там, где уже есть другие

системы, к которым пользователи привыкли. Возможными источниками такой путаницы могут быть:

- а) использование одинаковых команд (или сокращений) для обозначения различных действий;
- б) нестандартное использование клавиш специального назначения или других клавиш;
- в) различие в размещении данных на экране монитора.

6. Система должна быть гибкой, особенно, если она создаётся для длительного использования. !При этом необходимо оговорить, что стоимость системы окажется выше, чем первоначально ожидалось, так как её модернизация потребует дополнительных расходов.

7. Если создаваемая система при работе создаёт собственные системы, то должен осуществляться контроль за ресурсами.

5.1. ПРОЕКТИРОВАНИЕ ИНТЕРАКТИВНЫХ СИСТЕМ

Этапы проектирования интерактивной системы:

- 1) предложение новой системы;
- 2) постановка задачи;
- 3) детальная разработка системы;
- 4) тестирование системы;
- 5) внедрение системы.

Казалось бы, нет никаких различий от любой другой программной системы. Существенная разница кроется в том, что разрабатываемая система непосредственно используется людьми, не являющимися специалистами в области вычислительной техники, и предназначена

помочь им в их обычной работе. Поэтому будущих пользователей нужно привлекать к процессу разработки, но они не могут представить проектируемую систему, не зная способа, с помощью которого будет вводиться информация и выводиться конечный результат. Другими словами, тип диалога в интерактивной системе должен быть обозначен в общих чертах на самой ранней стадии разработки. На практике наиболее подходящий тип выбирается с учётом целей диалога и вида пользователей.

Перечислим различные типы

диалога: – выбор из меню;

- вопрос—ответ; – заполнение бланков:
- с произвольным порядком заполнения;
- с указанием порядка заполнения автоматическим перемещением курсора;
- ”прямой режим” с использованием языка команд.

При организации диалога используются различные режимы вывода информации на экран монитора:

- режим “весь экран” (при этом после каждого ответа изменяется вся информация на экране или добавляется информация в определённое место экрана);
- режим ”строка за строкой” (на экран выводится одна строка и после каждого шага диалога одна строка изменяется на другую, при этом вся информация на экране стирается и новая строка появляется в определённом месте экрана);
- переход от одной страницы к другой (очередная вводимая или выводимая строка появляется на экране внизу и одновременно теряется одна строка вверху; недостаток этого варианта в том, что информация на экране кажется непрерывно движущейся, так что данные трудно рассмотреть; с другой стороны, всегда видна последняя часть диалога);
- редактирование содержимого экрана (это своего рода экранные редакторы, позволяющие изменять информацию на экране).

5.2. ДИАЛОГ НА ОСНОВЕ ВЫБОРА ИЗ МЕНЮ

В такой системе пользователю предоставляется список вариантов выбора, возможных в данном месте диалога. Выбор приводит к появлению на экране либо последующих меню, либо к появлению требуемой информации, либо к переводу терминала в состояние, в котором можно вводить данные.

Общепринято, что системы, основанные на выборе меню, являются подходящими для новичков и случайных пользователей по двум причинам:

1) обычно наряду со списком действий, которые нужно предпринять, имеется достаточное число инструкций;

2) одно меню похоже на другое и это придаёт уверенности пользователю.

Но такие системы не лишены и недостатков:

1) такой тип диалога может показаться скучным для опытных пользователей;

2) они требуют больше компьютерных ресурсов, чем системы, использующие менее многословные типы диалога.

Режим "весь экран" больше подходит для меню, так как он допускает широкое использование инструкций и других средств помощи пользователю.

Применение меню уменьшает число символов, которые печатаются пользователем; ввод, таким образом, становится совершенно незатруднительным даже для неопытных работников. Ввод может быть упрощён до такой степени, что не всегда требуется клавиатура в полном объёме.

Не следует забывать, что средства, которые помогают пользователю, такие как указание выбора или напоминание о возможностях выбора на очередном шаге, могут действовать раздражающе на более опытного специалиста. Если система должна обслуживать и тех и других, можно предусмотреть альтернативные варианты. Это вызывает дополнительные затраты и необходимо заранее решить, окупятся ли они.

Меню могут существовать внутри других типов диалога. В частности, они могут появляться внутри диалога "вопрос—ответ" в момент, когда нужно перейти

по одному из нескольких направлений. Это позволит сократить число вопросов.

5.3. ДИАЛОГ ТИПА "ВОПРОС–ОТВЕТ"

Диалог 'вопрос–ответ' ведётся точно так, как подразумевает название: система задаёт вопрос, пользователь даёт ответ.

Достоинства такого типа диалога:

- 1) пригоден для неопытных пользователей;
- 2) лёгок при обучении.

Не лишен такой диалог и недостатков:

- 1) неудобен, если требуется неоднозначный или обширный ответ;
- 2) диалог должен соответствовать каждому конкретному приложению;
- 3) интерактивная система с таким диалогом лишена гибкости, так как структура диалога зафиксирована.

При проектировании вопросно-ответных систем необходимо:

- 1) чётко определить логическую структуру диалога;
- 2) исключить возможность повторяющихся вопросов;
- 3) чётко сформулировать вопросы, так, чтобы они допускали только однозначные ответы;
- 4) исключить лишние вопросы, не имеющие отношения к теме диалога;
- 5) продумать логичное размещение вопросов на экране;
- 6) продумать возможность вывода и даже редактирования протокола диалога.

5.4. ДИАЛОГ ТИПА ”ЗАПОЛНЕНИЕ БЛАНКА“

Такой диалог предполагает, что на экране появляется так называемый ”бланк”. Пользователь вводит сообщение в поля бланка. При проектировании такого типа диалога необходимо:

- 1) выбрать способ перемещения курсора по полям бланка:
 - автоматическое перемещение (после окончания ввода поля курсор перемещается в другое поле по заранее заданному маршруту);
 - произвольное перемещение (пользователь сам задаёт маршрут перемещения курсора);
- 2) продумать расположение полей на экране, в том числе и их логический порядок;
- 3) оценить количество информации и предусмотреть, если необходимо, переход от одного окна к другому.

5.5. ДИАЛОГ ТИПА ”ПРЯМОЙ РЕЖИМ“

Такой тип диалога предполагает наличие языка команд. Для его организации пользователь вводит некоторую команду с необходимыми параметрами и ключами.

При проектировании диалога типа “прямой режим” необходимо:

- 1) продумать грамматику языка команд;
- 2) предусмотреть, если возможно, использование символов одного регистра в одной команде;
- 3) предусмотреть возможность сокращения названия команд и их параметров;
- 4) предусмотреть однотипность названия команд, чтобы не возникало сложностей при переходе от одной команды к другой.

5.6. СРЕДСТВА ПОМОЩИ И ПОДДЕРЖКИ

Любая интерактивная система должна предоставлять пользователю возможность получить помощь, а при необходимости и поддержку. Поэтому разработчик вычислительной системы должен заранее предусмотреть способы их организации:

1) при вводе информации в диалоге необходимо провести проверку достоверности вводимых данных;

2) выводить сообщения о допущенных в процессе диалога ошибках;

3) при необходимости предусмотреть возможность вызова помощи. Для этого можно:

– нажать функциональную клавишу (например, F1); – ввести определённые символы (например, HELP); – использовать специальное поле для ввода символа помощи; 4) предусмотреть поддержку:

— неодушевлённую (документы, справочники, специальную литературу); – одушевлённую (специалист, который сопровождает систем